

CHAPTER 07

필터링

OpenCV 4로 배우는 컴퓨터 비전과 머신 러닝

7장 필터링

7.1 영상의 필터링

7.2 블러링: 영상 부드럽게 하기

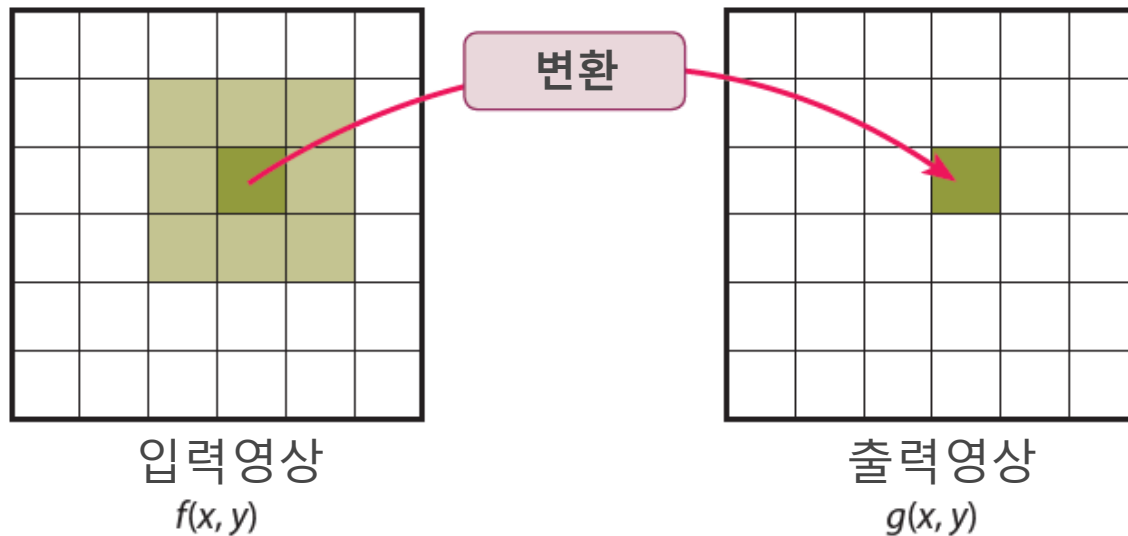
7.3 샤프닝: 영상 날카롭게 하기

7.4 잡음 제거 필터링

7.1 영상의 필터링

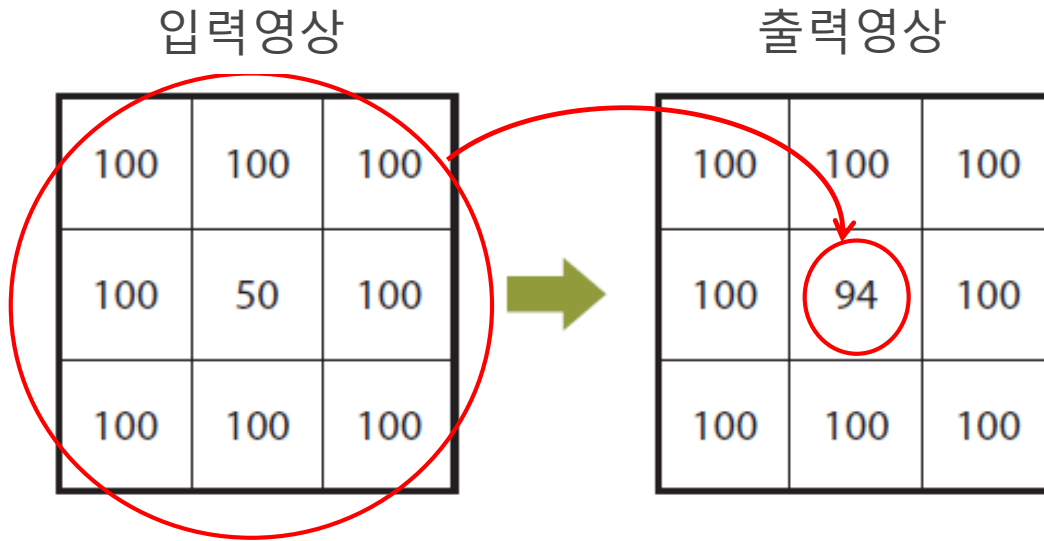
영상의 필터링(filtering)

- 영상에서 원하는 정보만 통과시키고 원치 않는 정보(잡음, 노이즈)는 걸러내는 작업
- 영상에서 지저분한 잡음(노이즈, noise)을 걸러 내어 영상을 깔끔하게 만들거나 또는 부드러운 느낌의 성분을 제거함으로써 영상을 좀 더 날카로운 느낌이 나도록 만들 수도 있음
- 인접 화소들의 값을 참조하여 화소의 값을 변경하는 처리



영상의 필터링

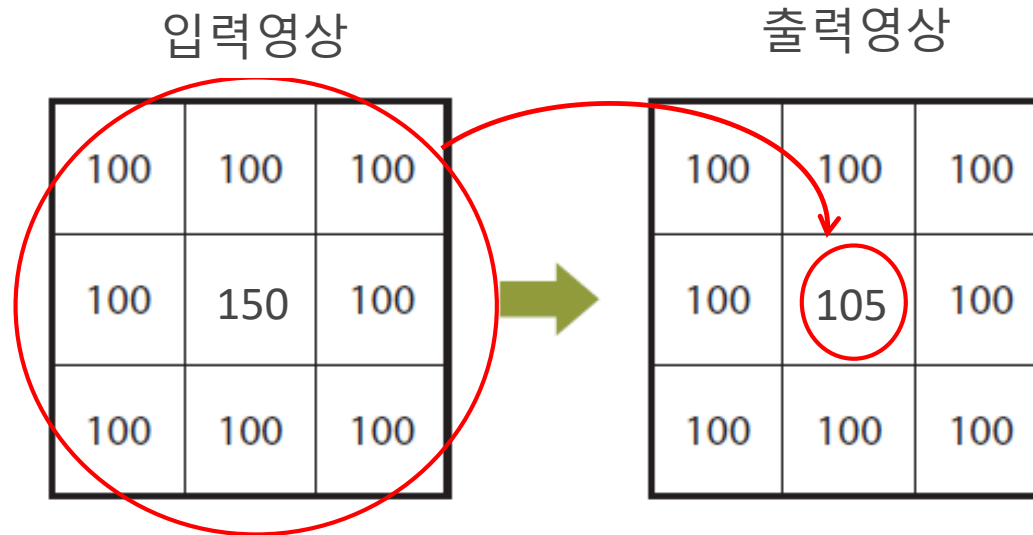
- 예 : 영상의 화소값을 인접 화소들의 평균값으로 교체 -> 주변보다 매우 작은 값(노이즈)들을 제거할 수 있다.



$$(100 + 100 + 100 + 100 + 50 + 100 + 100 + 100 + 100) / 9 = 94.4$$

영상의 필터링

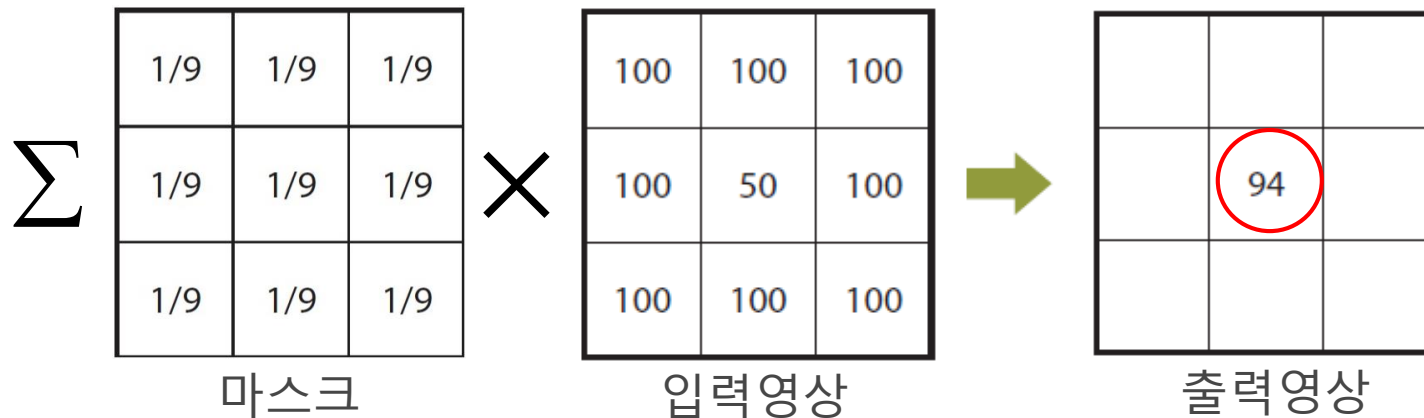
- 예 : 영상의 화소값을 인접 화소들의 평균값으로 교체 -> 주변보다 매우 큰 값(노이즈)들을 제거할 수 있다.



$$(100 + 100 + 100 + 100 + 150 + 100 + 100 + 100 + 100) / 9 = 105.5$$

영상의 필터링

- 영상의 필터링은 보통 마스크(mask)라고 부르는 작은 크기의 행렬을 이용하여 계산함 -> 출력영상 = Σ (마스크X입력영상)
- 마스크는 커널(kernel), 윈도우(window), 필터(filter)라고도 부름
- 마스크는 다양한 크기와 모양으로 정의할 수 있으며, 마스크 행렬의 원소는 보통 실수로 구성됨



$$(100 + 100 + 100 + 100 + 50 + 100 + 100 + 100 + 100) / 9 = 94.4$$

영상의 필터링

- 1×3 또는 3×1 형태의 직사각형 행렬 또는 3×3 , 5×5 등 정방형 행렬을 사용, 십자가 모양의 마스크를 사용할 수도 있음
- 3×3 정방형 행렬이 다양한 필터링 연산에서 가장 널리 사용됨
- 그림 7-1에 표시한 다양한 마스크에서 진한 색으로 표시한 위치는 고정점(anchor point)을 나타냄
- 고정점은 현재 필터링 작업을 수행하고 있는 기준 픽셀 위치를 나타내고, 대부분의 경우 마스크 행렬 정중앙을 고정점으로 사용함

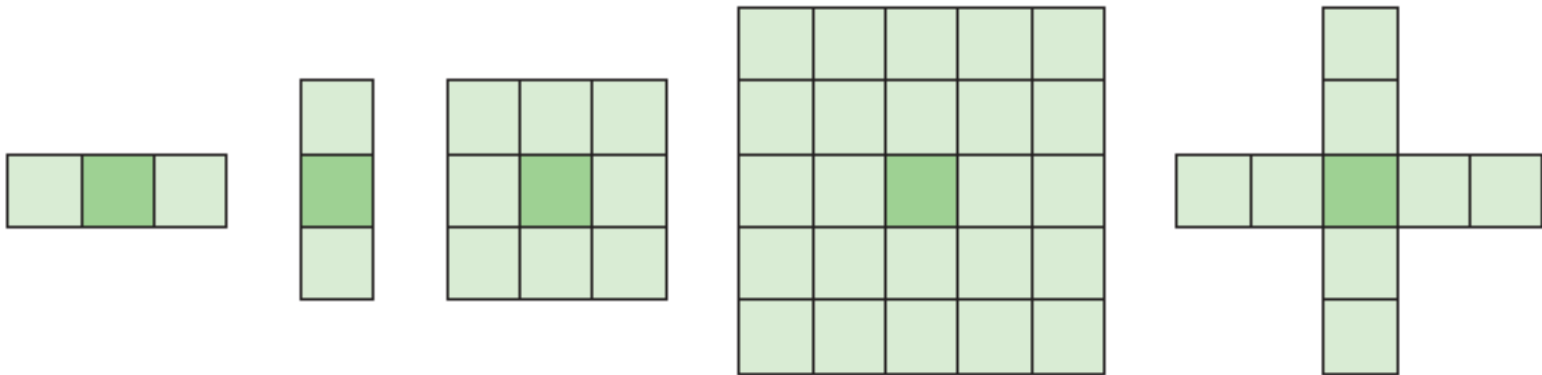


그림 7-1 다양한 필터 마스크 모양

영상의 필터링

- 필터링 연산의 결과는 마스크 행렬의 모양과 원소 값에 의해 결정됨
- 마스크 행렬을 어떻게 정의하는가에 따라 영상을 부드럽게 만들 수도 있고, 반대로 날카롭게 만들 수도 있고, 잡음을 제거하거나 에지(edge) 성분만 나타나도록 만들 수도 있음
- 마스크를 이용한 필터링은 입력 영상의 모든 픽셀 위로 마스크 행렬을 이동시키면서 마스크 연산을 수행하는 방식으로 이루어짐
- 마스크 연산이란 마스크 행렬의 모든 원소에 대하여 마스크 행렬 원소 값과 같은 위치에 있는 입력 영상 픽셀 값을 서로 곱한 후, 그 결과를 모두 더하는 연산임
- 마스크 연산의 결과를 출력 영상에서 고정점 위치에 대응되는 픽셀 값으로 설정함

영상의 필터링

- 마스크 행렬 m 의 고정점이 입력 영상의 (x, y) 좌표 위에 위치했을 때 필터링 결과 영상의 픽셀 값 $g(x, y)$ 는 다음과 같이 계산됨

$$g(x, y) = m(0,0)f(x-1, y-1) + m(1,0)f(x, y-1) + m(2,0)f(x+1, y-1) \\ + m(0,1)f(x-1, y) + m(1,1)f(x, y) + m(2,1)f(x+1, y) \\ + m(0,2)f(x-1, y+1) + m(1,2)f(x, y+1) + m(2,2)f(x+1, y+1)$$

출력영상의 좌표 (x, y) ,
입력영상의 좌표 (x, y) ,
마스크의 고정점이
일치하도록 배치

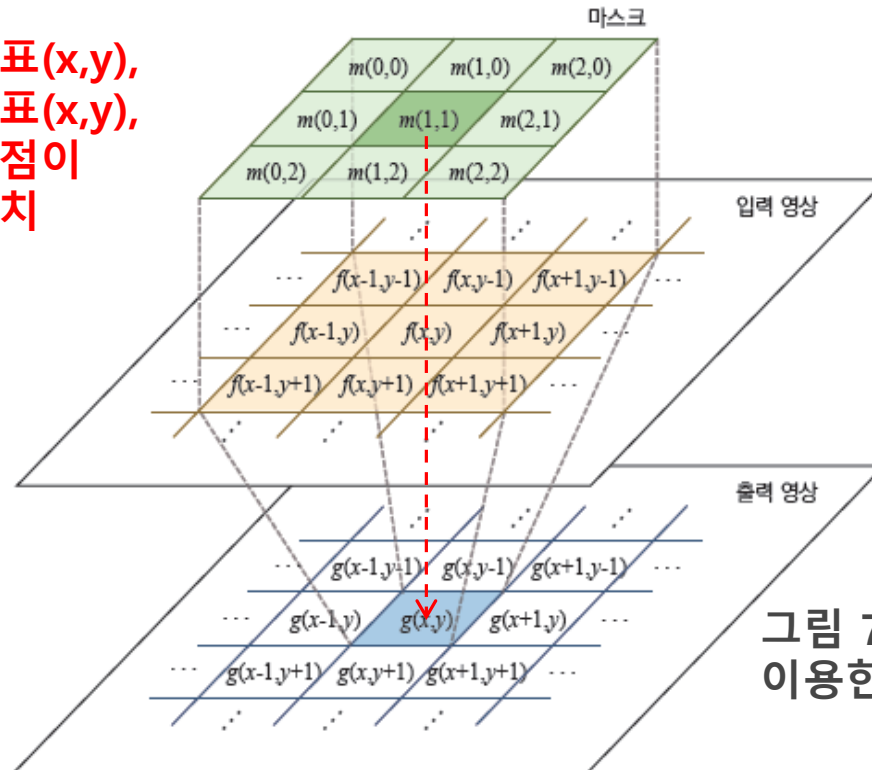


그림 7-2 마스크를
이용한 필터링 연산 방법

영상의 필터링

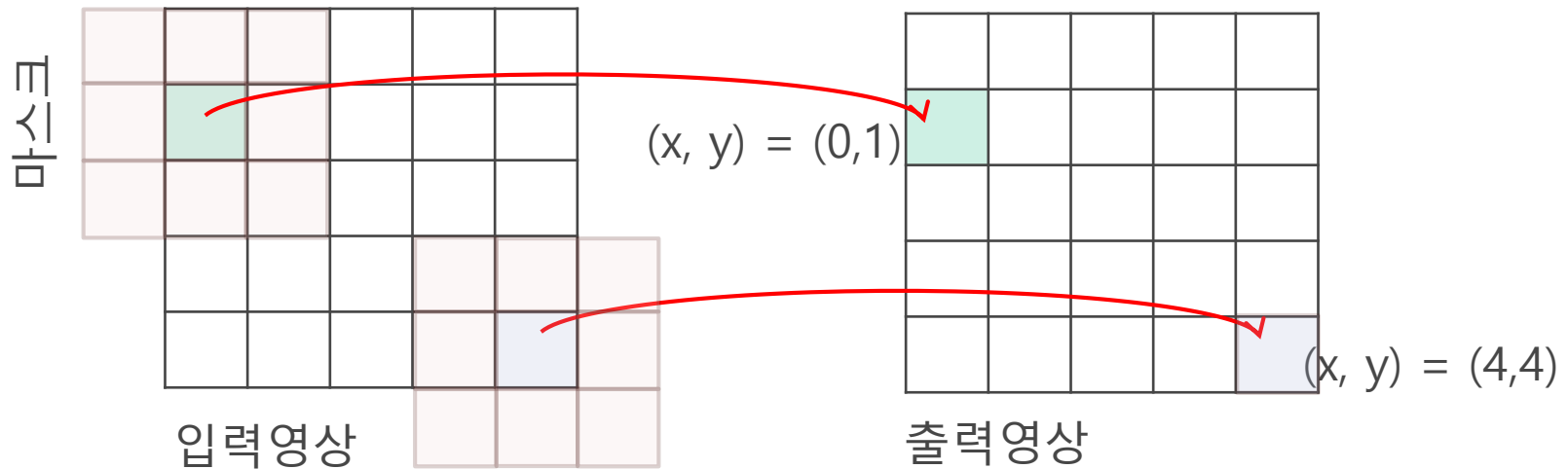
- 영상의 가장자리 픽셀이란 영상에서 가장 왼쪽 또는 오른쪽 열, 가장 위쪽 또는 아래쪽 행에 있는 픽셀을 의미함
- 예를 들어 $(x, y) = (0, 1)$ 위치에서 3×3 크기의 마스크 연산을 수행하는 경우, 결과 영상의 픽셀 값 $g(0, 1)$ 은 다음과 같이 계산됨

$$\begin{aligned} g(0, 1) = & m(0, 0)f(-1, 0) + m(1, 0)f(0, 0) + m(2, 0)f(1, 0) \\ & + m(0, 1)f(-1, 1) + m(1, 1)f(0, 1) + m(2, 1)f(1, 1) \\ & + m(0, 2)f(-1, 2) + m(1, 2)f(0, 2) + m(2, 2)f(1, 2) \end{aligned}$$

- 앞 수식에서 $x = -1$ 인 위치에서의 픽셀 값, 즉 $f(-1, 0)$, $f(-1, 1)$, $f(-1, 2)$ 세 픽셀은 실제 영상에 존재하지 않음
- 이 수식은 적용할 수 없으며, 영상의 가장자리 픽셀에 대해 필터링을 수행할 때에는 특별한 처리를 해야 함

영상의 필터링

- 영상의 가장자리 픽셀을 처리할때 마스크에 대응되는 입력 픽셀이 존재하지 않는다. -> 특별한 처리 필요



영상의 필터링

- OpenCV는 영상의 필터링을 수행할 때, 영상의 가장자리 픽셀을 확장하여 영상 바깥쪽에 가상의 픽셀을 만듦
- 영상의 바깥쪽 가상의 픽셀 값을 어떻게 설정하는가에 따라 필터링 연산 결과가 달라짐
- 그림 7-3은 입력 영상의 좌측 상단 부분을 확대하여 나타낸 것으로 각각의 사각형은 픽셀을 표현함
- 실선으로 그려진 노란색 픽셀은 영상에 실제 존재하는 픽셀이고, 점선으로 표현된 바깥쪽 분홍색 픽셀은 필터링 연산 시 사용할 가상의 픽셀임
- 이 그림에서는 5×5 크기의 필터 마스크를 사용하는 필터링을 고려하여 영상 바깥쪽에 두 개씩의 가상 픽셀을 표현함
- OpenCV는 이러한 가장자리 픽셀 확장 방법을 이용하여 영상의 가장자리 픽셀에 대해서도 문제없이 필터링 연산을 수행함

영상의 필터링

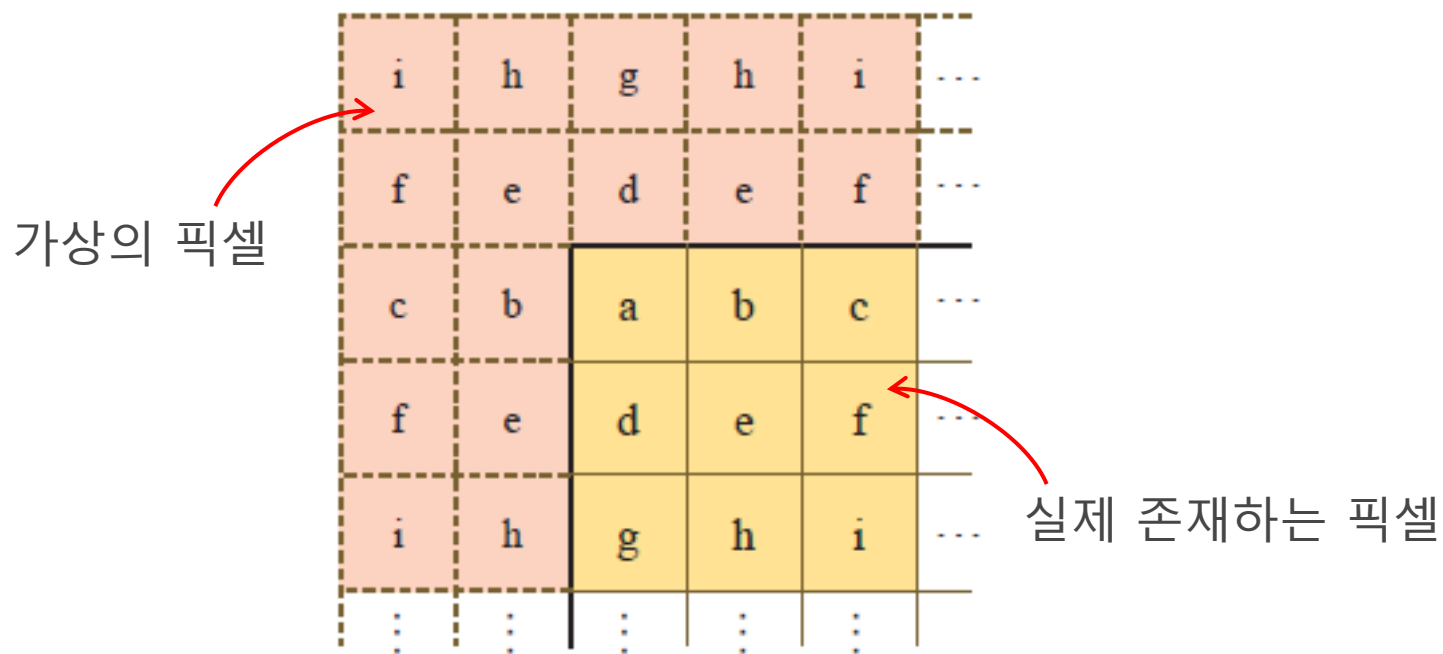


그림 7-3 필터링 연산을 위한 기본적인 가장자리 픽셀 확장 방법

영상의 필터링

- 표 7-1에 나타난 상수는 BorderTypes라는 이름의 열거형 상수 중 일부임

표 7-1 OpenCV 필터링에서 가장자리 픽셀 처리 방법

BorderTypes 열거형 상수	설명														
BORDER_CONSTANT	<table><tr><td>0</td><td>0</td><td>0</td><td>a</td><td>b</td><td>c</td><td>d</td><td>e</td><td>f</td><td>g</td><td>h</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	a	b	c	d	e	f	g	h	0	0	0
0	0	0	a	b	c	d	e	f	g	h	0	0	0		
BORDER_REPLICATE	<table><tr><td>a</td><td>a</td><td>a</td><td>a</td><td>b</td><td>c</td><td>d</td><td>e</td><td>f</td><td>g</td><td>h</td><td>h</td><td>h</td><td>h</td></tr></table>	a	a	a	a	b	c	d	e	f	g	h	h	h	h
a	a	a	a	b	c	d	e	f	g	h	h	h	h		
BORDER_REFLECT	<table><tr><td>c</td><td>b</td><td>a</td><td>a</td><td>b</td><td>c</td><td>d</td><td>e</td><td>f</td><td>g</td><td>h</td><td>h</td><td>g</td><td>f</td></tr></table>	c	b	a	a	b	c	d	e	f	g	h	h	g	f
c	b	a	a	b	c	d	e	f	g	h	h	g	f		
BORDER_REFLECT_101	<table><tr><td>d</td><td>c</td><td>b</td><td>a</td><td>b</td><td>c</td><td>d</td><td>e</td><td>f</td><td>g</td><td>h</td><td>g</td><td>f</td><td>e</td></tr></table>	d	c	b	a	b	c	d	e	f	g	h	g	f	e
d	c	b	a	b	c	d	e	f	g	h	g	f	e		
BORDER_REFLECT101	BORDER_REFLECT_101과 같음														
BORDER_DEFAULT	BORDER_REFLECT_101과 같음														

영상의 필터링

```
void filter2D(InputArray src, OutputArray dst, int ddepth,
              InputArray kernel, Point anchor = Point(-1,-1),
              double delta = 0, int borderType = BORDER_DEFAULT);
```

- **src** 입력 영상
- **dst** 출력 영상. src와 같은 크기, 같은 채널 수를 갖습니다.
- **ddepth** 결과 영상의 깊이
- **kernel** 필터링 커널. 1채널 실수형 행렬
- **anchor** 고정점 좌표. Point(-1, -1)을 지정하면 커널 중심을 고정점으로 사용합니다.
- **delta** 필터링 연산 후 추가적으로 더할 값
- **borderType** 가장자리 픽셀 확장 방식

- OpenCV에서 마스크를 사용하는 일반적인 필터링은 filter2D 함수를 이용하여 수행함

영상의 필터링

- filter2D() 함수는 src 영상에 kernel(마스크)를 이용하여 필터링을 수행하고, 그 결과를 dst에 저장함
- filter2D() 함수가 수행하는 연산을 수식으로 표현하면 다음과 같음

$$dst(x, y) = \sum_j \sum_i kernel(i, j) \cdot src(x + i - anchor.x, y + j - anchor.y) + delta$$

- ddepth는 결과 영상의 깊이를 지정함, 만약 -1을 지정하면 출력 영상의 깊이는 입력 영상과 같게 설정됨

표 7-2 입력 영상의 깊이에 따라 지정 가능한 ddepth 값

입력 영상의 깊이(src.depth())	지정 가능한 ddepth 값
CV_8U	-1/CV_16S/CV_32F/CV_64F
CV_16U/CV_16S	-1/CV_32F/CV_64F
CV_32F	-1/CV_32F/CV_64F
CV_64F	-1/CV_64F

영상의 필터링

- `filter2D()` 함수에서 `anchor`, `delta`, `borderType` 인자는 기본값을 가지고 있기 때문에 생략할 수 있음
- `anchor` 인자는 커널(마스크) 행렬에서 고정점으로 사용할 좌표이며, 기본값인 `Point(-1, -1)`을 지정하면 커널 행렬 중심 좌표를 고정점으로 사용함
- `delta` 인자에는 필터링 연산 후 결과 영상에 추가적으로 더할 값을 지정할 수 있으며, 기본값은 0임
- `borderType` 인자에는 앞서 표 7-1에 나타낸 `BorderTypes` 열거형 상수 중 하나를 지정할 수 있음

엠보싱 필터링

- 엠보싱 필터는 입력 영상을 엠보싱(양각) 느낌이 나도록 변환하는 필터임
- 보통 입력 영상에서 픽셀 값 변화가 적은 평탄한 영역은 회색으로 설정함
- 객체의 경계 부분은 좀 더 밝거나 어둡게 설정하면 엠보싱 느낌이 남

-1	-1	0
-1	0	1
0	1	1

그림 7-4 엠보싱 필터 마스크

엠보싱 필터링

- 대각선 방향으로 픽셀 값이 급격하게 변하는 부분에서 결과 영상 픽셀 값이 0보다 훨씬 크거나 또는 0보다 훨씬 작은 값을 가지게 됨

$$\Sigma \begin{bmatrix} -1 & -1 & 0 \\ -1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \times \begin{bmatrix} 10 & 10 & 100 \\ 10 & 100 & 100 \\ 100 & 100 & 100 \end{bmatrix} = \begin{bmatrix} & & \\ & 270 & \\ & & \end{bmatrix}$$

$$\Sigma \begin{bmatrix} -1 & -1 & 0 \\ -1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \times \begin{bmatrix} 100 & 100 & 100 \\ 100 & 100 & 10 \\ 100 & 10 & 10 \end{bmatrix} = \begin{bmatrix} & & \\ & -270 & \\ & & \end{bmatrix}$$

엠보싱 필터링

- 입력 영상에서 픽셀 값이 크게 바뀌지 않는 평탄한 영역에서는 결과 영상의 픽셀 값이 0에 가까운 값을 가지게 됨

$$\Sigma \begin{bmatrix} -1 & -1 & 0 \\ -1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \times \begin{bmatrix} 100 & 100 & 100 \\ 100 & 100 & 100 \\ 100 & 100 & 100 \end{bmatrix} = \begin{bmatrix} & & \\ & 0 & \\ & & \end{bmatrix}$$

- 이렇게 구한 결과 영상을 그대로 화면에 나타내면 음수 값은 모두 포화 연산에 의해 0이 되어 버리기 때문에 엠보싱 효과 사라짐
- 엠보싱 필터를 구현할 때에는 결과 영상에 128을 더하는 것이 보기에 좋음 : 음수 -> 0, 0 -> 128, 양수 -> 255

코드7-1 : 엠보싱 필터링

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main(void)
{
    Mat src = imread("rose.bmp", IMREAD_GRAYSCALE);
    if (src.empty()) { cerr << "Image load failed!" << endl; return -1;}

    float data[ ] = { -1, -1, 0, -1, 0, 1, 0, 1, 1 };
    Mat emboss(3, 3, CV_32FC1, data);
    Mat dst;
    filter2D(src, dst, -1, emboss, Point(-1, -1), 128);

    imshow("src", src);
    imshow("dst", dst);
    waitKey();
    return 0;
}
```

코드 7-1 : 엠보싱 필터링

- dst 영상에서 장미꽃 경계 부분이 입체감 있게 표현된 것을 확인할 수 있음
- 픽셀 값이 완만하게 바뀌는 부분에서는 필터링 결과 영상이 대체로 밝기 값 128에 가까운 회색으로 표현됨

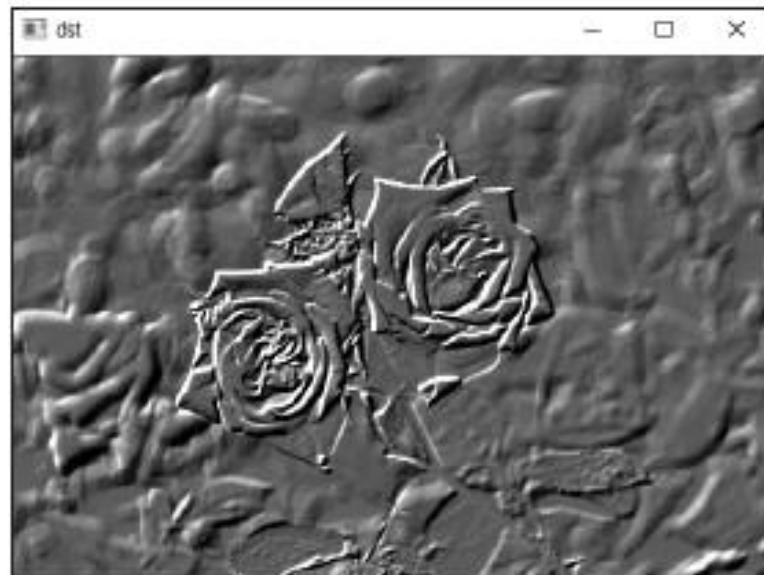


그림 7-5 엠보싱 필터링 예제 실행 결과

실습과제1

- 코드 7-1에서 필터를 아래 마스크로 바꾼 후 결과를 출력하라.
- 매개변수 $\text{delta} \Rightarrow 0$ 으로 설정하라.
- 아래 필터마스크는 어떤 역할을 수행하는가?

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

실습과제2

- 코드 7-1에서 필터를 아래 마스크로 바꾼 후 결과를 출력하라.
- 매개변수 $\text{delta} \Rightarrow 0$ 으로 설정하라
- 아래 필터마스크는 어떤 역할을 수행하는가?

-1	-1	-1
-1	9	-1
-1	-1	-1

실습과제3

- 코드 7-1에서 필터를 아래 마스크로 바꾼 후 결과를 출력하라.
- 매개변수 $\text{delta} \Rightarrow 0, 128$ 으로 설정하라
- 아래 필터마스크는 어떤 역할을 수행하는가?

-1	0	1
-2	0	2
-1	0	1

실습과제4(선택문제)

- 코드 7-1에서 입력영상을 출력하고 영상윈도우에서 마우스 왼쪽버튼을 클릭하면 실습과제 1번의 결과를 출력하고 마우스 우측버튼을 클릭하면 실습과제 2번의 결과를 출력하고 마우스를 더블 클릭하면 실습과제 3번의 결과가 출력되도록 하시오.
- 메인함수에서 먼저 `namedWindow`함수를 이용하여 결과를 출력할 윈도우를 먼저 생성하고 이벤트 핸들러에서는 `imshow`함수로 생성한 윈도우에 출력만 하시오.

과제 제출 방법

- 소스파일, 라인단위 코드설명, 실행결과를 하나의 PDF 파일로 작성하여 과제게시판에 제출하시오. 이외 양식은 0점 처리함
- 제출기한은 과제 게시판을 참고할 것.